PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Переобучение и проверка модели на новых данных

Содержание

- Вступление
- Краткая теория перед практикой
- Подготовка среды и данных
- Чек-лист самопроверки
- Что сдавать

Вступление

Привет! Представьте ученика, который выучил ответы на тренировочный вариант, но не понял способ решения. На знакомых вопросах он получает максимальный балл, а на новых — ошибается. С моделью машинного обучения может произойти то же самое. Сегодня Вы научитесь замечать такую ситуацию по результатам на обучающих и новых данных, а затем уменьшать переобучение модели.
Главная трудность состоит в том, что качество на данных, уже использованных при обучении, может создавать ложное ощущение успеха. Поэтому модель необходимо проверять на отдельной части набора, которую она не видела во время настройки. Такое сравнение показывает не только способность запоминать примеры, но и способность переносить найденные закономерности на новые объекты.

После занятия Вы сможете

разделять данные на обучающую и тестовую части; вычислять качество модели на обеих выборках; сравнивать результаты; распознавать признаки переобучения; объяснять, почему высокая точность на обучающих данных ещё не гарантирует полезность модели.

> **Основной вопрос занятия**

> Насколько хорошо модель решает задачу на объектах, которые не участвовали в её обучении? Ответ на этот вопрос важнее, чем максимальное значение качества на обучающей выборке.

Краткая теория перед практикой

1. Обучающая и тестовая выборки

Обучающая выборка используется для настройки модели: алгоритм ищет закономерности между признаками `X` и правильными ответами `y`. Тестовая выборка временно откладывается и применяется только после обучения. Поэтому она имитирует новые данные, которые система встретит после запуска.

При разделении исходного набора признаки и ответы должны оставаться согласованными: строке `X_train` соответствует ответ из `y_train`, а строке `X_test` — ответ из `y_test`. Модель обучают на паре `X_train` и `y_train`, а затем отдельно получают предсказания для обучающих и тестовых признаков.

Если тестовые объекты или их правильные ответы повлияли на обучение, оценка перестаёт быть независимой. Такая ситуация называется утечкой данных: модель получает информацию, которой у неё не должно быть при работе с действительно новыми объектами.

> **Важно**
>
> Тестовая выборка предназначена для проверки, а не для настройки модели. Нельзя обучать модель на `X_test` и `y_test`, а затем представлять результат на этих же данных как качество на новых объектах.

2. Что такое переобучение

Переобучение — ситуация, когда модель слишком точно подстроилась под обучающие примеры, включая случайный шум и единичные особенности. Она хорошо воспроизводит уже виденные данные, но хуже работает на новых.

Для дерева решений риск переобучения обычно возрастает при чрезмерном усложнении структуры. Глубокое дерево может создавать множество узких правил, подходящих только для небольших групп обучающих объектов. Ограничение глубины и других параметров сложности помогает сосредоточиться на более общих закономерностях.

Однако высокая обучающая точность сама по себе ещё не доказывает переобучение. Диагноз появляется только после сравнения с качеством на независимых данных: если обе оценки

высоки и близки, модель может успешно обобщать найденные закономерности.

Бытовая аналогия
Модель похожа на ученика, который запомнил конкретные ответы, но не освоил правило. На знакомом варианте результат почти идеален, а новая формулировка вызывает затруднение.

3. Как заметить переобучение

Для первичной диагностики вычисляют одну и ту же метрику на двух выборках. В этом занятии используется `accuracy` — доля объектов, для которых модель правильно определила класс.

$$\text{accuracy} = \text{количество правильных предсказаний} / \text{общее количество предсказаний}$$

Значение `accuracy` находится в диапазоне от 0 до 1. Например, результат 0,80 означает, что модель дала правильный ответ для 80% проверенных объектов. Для сопоставления оценок рассчитывается разрыв:

$$\text{gap} = \text{train_accuracy} - \text{test_accuracy}$$

Положительный и большой разрыв означает, что на знакомых данных модель работает заметно лучше, чем на новых. При этом важно учитывать не только разрыв, но и сами значения метрики.

Наблюдение	Возможная интерпретация
Высокое качество на `train` и заметно более низкое на `test`	Есть признаки переобучения
Низкое качество и на `train`, и на `test`	Модель может быть слишком простой или данные недостаточно информативны
Сопоставимое приемлемое качество на `train` и `test`	Явных признаков переобучения нет

Важно: универсального порога разницы нет. В этом практикуме для учебной диагностики используется условный порог 0,10, но в реальном проекте допустимый разрыв зависит от задачи, объёма данных, выбранной метрики и цены ошибки.
Небольшой разрыв также не гарантирует хорошую модель. Например, близкие, но низкие значения на обеих выборках могут указывать на то, что модель не смогла обнаружить полезную закономерность. Поэтому формулировка «переобучения не видно» не равна формулировке «модель имеет высокое качество».

> **Частая ошибка**
>
> Нельзя сравнивать разные метрики, например `accuracy` на обучающей выборке и другую метрику на тестовой. Для вычисления разрыва метод оценки и условия проверки должны быть одинаковыми.

4. Решённый мини-пример

Модель получила `accuracy` 0,98 на обучающей выборке и 0,71 на тестовой. Разрыв равен `0,98 − 0,71 = 0,27`. Обучающий результат очень высокий, а разрыв велик. Это сильный сигнал переобучения: модель хорошо запомнила обучающие данные, но значительно хуже переносит найденные правила на новые объекты.
Для сравнения рассмотрим другой результат: `train accuracy = 0,84`, `test accuracy = 0,81`. Разрыв составляет 0,03. По учебному порогу явных признаков переобучения нет, хотя окончательная оценка полезности модели всё равно зависит от требований конкретной задачи.

> **Мини-проверка**
>
> Если `train accuracy = 0,63`, а `test accuracy = 0,61`, можно ли назвать модель качественной только потому, что разрыв равен 0,02? Обоснуйте ответ, учитывая оба значения метрики.

Подготовка среды и данных

1. Установка библиотек

В Google Colab библиотеки обычно уже установлены. В локальной среде выполните команду в терминале:
Команда установки

```
pip install scikit-learn pandas matplotlib
```

Библиотека `scikit-learn` потребуется для создания данных, разделения выборки, обучения дерева решений и вычисления метрики. `pandas` удобно использовать для оформления результатов, а `matplotlib` — для построения графиков.

Правило выполнения

Запускайте ячейки по порядку. Не переобучайте модель на тестовой выборке и не меняйте `random_state=42` до выполнения обязательной части: это нужно для воспроизводимости результатов.

Фиксированное значение `random_state` позволяет повторно получить одинаковую генерацию и одинаковое случайное разделение данных. Благодаря этому различия между моделями можно связывать с их параметрами, а не со случайно изменившимся составом выборок.

2. Импорт инструментов

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

После выполнения ячейки импортированные инструменты становятся доступны в следующих ячейках блокнота. Если среда была перезапущена, импорт и создание данных необходимо выполнить заново до обращения к переменным и моделям.

3. Учебный набор данных

Мы создадим синтетический набор для бинарной классификации. Каждая строка описывает объект двадцатью числовыми признаками. Целевая переменная принимает значение 0 или 1. Часть признаков действительно связана с классом, а часть содержит избыточность и шум. Это позволяет увидеть переобучение без скачивания внешнего файла.

```
X, y = make_classification(
n_samples=600,
n_features=20,
n_informative=5,
n_redundant=5,
n_repeated=0,
n_classes=2,
flip_y=0.12,
class_sep=0.8,
random_state=42,
)
```

Параметр `n_samples=600` задаёт число объектов, а `n_features=20` — число признаков каждого объекта. Из них пять признаков являются информативными, ещё пять создаются как избыточные, а остальные не несут такой же прямой полезной информации для разделения классов.
Параметр `flip_y=0.12` добавляет шум в целевые значения, а `class_sep=0.8` управляет разделимостью классов. Наличие шума делает учебную задачу подходящей для наблюдения за тем, как слишком сложная модель подстраивается под отдельные обучающие примеры.

> **Проверьте данные перед обучением**
>
> Убедитесь, что число строк в `X` совпадает с числом ответов в `y` . После разделения сумма размеров обучающей и тестовой частей должна равняться исходным 600 объектам.

4. Честное разделение данных

Разделите набор до обучения любой модели. Параметр `stratify=y` помогает сохранить примерно одинаковое соотношение классов 0 и 1 в обучающей и тестовой частях. Это особенно важно, когда классы представлены неравномерно.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y,
)
```

При `test_size=0.30` тестовая часть содержит 30% объектов, а обучающая — оставшиеся 70%. Для 600 объектов это 180 тестовых и 420 обучающих примеров. Эти размеры можно проверить через атрибут `shape`.

```
print(X_train.shape, X_test.shape)
print(y_train.shape, y_test.shape)
```

Ожидаемые размеры признаков: `(420, 20)` для `X_train` и `(180, 20)` для `X_test`. Целевые массивы должны содержать соответственно 420 и 180 значений.

Чек-лист самопроверки

- ☐ Я использовал одну и ту же разбивку данных для сравниваемых моделей.
- ☐ Я обучал модели только на `X_train` и `y_train`.
- ☐ Я посчитал `accuracy` отдельно для `train` и `test`.
- ☐ Я вычислил разрыв `train_accuracy - test_accuracy`.
- ☐ Я не сделал вывод о переобучении только по одному показателю.
- ☐ Моя функция диагностики проходит все `assert`-проверки.
- ☐ На графике есть подписи осей, легенда и заголовок.
- ☐ В выводе объяснено, почему переобучение опасно на новых данных.
- ☐ Число предсказаний совпадает с числом правильных ответов в каждой вычисляемой метрике.
- ☐ Я сохранил фиксированное значение `random_state` в обязательной части.
- ☐ Я проверил порядок выполнения ячеек после перезапуска среды.
- ☐ В итоговом выводе приведены конкретные значения метрик и разрыва.

Финальная мини-проверка

Ответьте одним предложением: почему результат `accuracy = 1.0` на обучающей выборке нельзя использовать как единственное доказательство качества модели?

Что сдавать

- Файл `.ipynb` или `.py` с выполненными обязательными заданиями.
- Заполненную итоговую таблицу с показателями трёх моделей.
- График зависимости `accuracy` от глубины дерева.
- Итоговый вывод объёмом 5–7 предложений.

Перед отправкой выполните файл целиком в правильном порядке и убедитесь, что все используемые переменные создаются в самом решении. В блокноте должны быть видны результаты вычисления метрик и построенный график, а названия моделей в таблице должны соответствовать моделям в коде.

Ключевая мысль урока

Качество модели оценивают не по тому, насколько хорошо она запомнила обучение, а по тому, насколько надёжно она работает на новых данных.

Итоговый ориентир

Хороший эксперимент разделяет обучение и проверку, использует одинаковые условия для сравнения моделей и подтверждает вывод несколькими связанными показателями: обучающей точностью, тестовой точностью и разрывом между ними.